



KJ's Educational Institute
TRINITY ACADEMY OF ENGINEERING, PUNE
(Approved by AICTE, New Delhi, Govt. of Maharashtra & affiliated to
SPPU, DTE Code: EN6634)
(Accredited by NAAC with 'A+' Grade)



DEPARTMENT OF COMPUTER ENGINEERING



Lab Manual

Data Structures lab

PCC-201-COM

AY-2025-2026

SE- SEM-1

Prepared By - **Prof.U.N.Yadav**



KJ's Educational Institute
TRINITY ACADEMY OF ENGINEERING, PUNE

Practical Plan A.Y.2025-26 Semester-1



Department of Computer Engineering				
Class: SE			Subject: Data Structures Laboratory	
Name of Staff : U.N.Yadav			Code:	PCC-201-COM
Examination Scheme:Term Work : 50 Marks Practical : 25 Marks				
Experiment No.	Title of Assignment/Experiment	Batch	Planned Date	Conducte d Date
Data Structures LAB				
1	Write a Python program to manage the borrowing records of books in a library. Implement the following functionalities: • Compute the average number of books borrowed by all library members. • Find the book with the highest and lowest number of borrowings in the library. • Count the number of members who have not borrowed any books (denoted by a borrow count of 0). • Display the most frequently borrowed book (i.e., the mode of borrow counts). After performing, determine the time and Space complexity of each operation.	S1		
		S2		
2	In an e-commerce system, customer account IDs are stored in a list. Write a program to implement the following operations: Linear Search: Check if a particular customer account ID exists in the list. Binary Search: Improve the search efficiency by implementing binary search to find if a customer account ID exists (on a sorted list).	S1		
		S2		
3	3 In a company, employee salaries are stored in a list as floating-point numbers. Write a Python program	S1		

	that sorts the employee salaries in ascending order using the following two algorithms: •Selection Sort: Sort the salaries using the selection sort algorithm. •Bubble Sort: Sort the salaries using the bubble sort algorithm. After sorting the salaries, the program should display top five highest salaries in the company.	S2		
4	B-1 Implementing a real-time undo/redo system for a text editing application using a Stack data structure. The system should support the following operations: • Make a Change: A new change to the document is made. • Undo Action: Revert the most recent change and store it for potential redo. • Redo Action: Reapply the most recently undone action. • Display Document State: Show the current state of the document after undoing or redoing an action	S1		
		S2		
5	B-2 Implement a real-time event processing system using a Queue data structure. The system should support the following features: • Add an Event: When a new event occurs, it should be added to the event queue. • Process the Next Event: The system should process and remove the event that has been in the queue the longest. • Display Pending Events: Show all the events currently waiting to be processed. • Cancel an Event: An event can be canceled if it has not been processed.	S1		
		S2		
6	A call center receives incoming calls, and each call is assigned a unique customer ID. The calls are answered in the order they are received. Your task is to simulate the call queue of a call center using a queue data structure. addCall(customerID, callTime): Add a call to the queue with the customer ID and the call time (in minutes). • answerCall(): Answer and remove the first call from the queue.	S1		
		S2		

	• viewQueue(): View all calls currently in the queue without removing them. • isEmpty(): Check if the queue is empty.			
7	C-1 Implement a hash table of size 10 and use the division method as a hash function. In case of a collision, use chaining. Implement the following operations: • Insert(key): Insert key-value pairs into the hash table. • Search(key): Search for the value associated with a given key. • Delete(key): Delete a key-value pair from the hash table	S1		
		S2		
8	C-2 Design and implement a hash table of fixed size. Use the division method for the hash function and resolve collisions using linear probing. Allow the user to perform the following operations: • Insert a key • Search for a key • Delete a key • Display the table	S1		
		S2		
9	D-1 Consider a particular area in your city. Note the popular locations A, B, C . . . in that area. Assume these locations represent nodes of a graph. If there is a route between two locations, it is represented as connections between nodes. Find out the sequence in which you will visit these locations, starting from (say A) using (i) BFS and (ii) DFS. Represent a given graph using an adjacency matrix to perform DFS and an adjacency list to perform BFS	S1		
		S2		
10	Implement various operations on a Binary Search Tree, such as insertion, deletion, display, and search..	S1		
		S2		
11	D-4 Construct an expression tree from the given prefix expression, e.g., $+-a*bc/def$, traverse it using post-order traversal (non-recursive), and then delete the entire tree.	S1		
		S2		

GROUP – E(Mini Project)**Note : You can implement any two Mini Project from DSL Syllabus**

12	Implementation of Snake and Ladder [BFS]				
13	Implementation of Flight Reservation System [Searching and Sorting]				

Subject Incharge

HOD

Instructions for writing journals

- 1) Attach front page containing Experiment No. , Title, and Performed date for every experiment. Specimen copy of the format for front page is given.
- 2) Write theory in detail from the points provided in the lab manuals.
- 3) Diagrams must be drawn on blank page with pencil only, do not use pen to draw diagram.
- 4) Use scales wherever necessary do not use free hand drawing.
- 5) Do not copy theory from manuals as it is in brief and given only for reference. Write your own theory.
- 6) Always attach conclusion at last.

Group A-Experiment No: 01	
Title: Python Program to Manage Borrowing Records in a Library.	
Date of performance:	Date of Submission:
Class: SE	Roll No.:
Signature:	

Experiment No. 1

Title: Write a Python program to manage the borrowing records of books in a library. Implement the following functionalities:

- Compute the average number of books borrowed by all library members.
- Find the book with the highest and lowest number of borrowings in the library.
- Count the number of members who have not borrowed any books (denoted by a borrow count of 0).
- Display the most frequently borrowed book (i.e., the mode of borrow counts).

After performing, determine the time and Space complexity of each operation

Objective:

- To develop a program that handles borrowing data for a library.
- To practice use of Python lists/dictionaries and functions.
- To analyze and compute average, maximum, minimum, count, and mode from data.
- To understand time and space complexity of implemented operations.

Outcomes:

- Students will be able to handle real-world data problems using Python.
- Students will learn how to analyze frequency and statistical operations on data.
- Students will be able to apply complexity analysis.

Software and Hardware Requirements:

- **Software:** Python 3.10+, Jupyter Notebook or any IDE (VS Code / PyCharm)
- **Hardware:** Minimum 2 GB RAM, 1 GHz processor, Any OS (Windows/Linux)

Theory:

The problem involves data representation and statistical analysis using programming. Lists or dictionaries can store borrow data. Various aggregate functions such as average, min, max, mode, and count are used to evaluate the dataset. Time and space complexity are evaluated based on the nature of operations like iteration, lookup, and insertion.

Mathematical Model :

Let

$M = \{\text{set of all members}\}$

$B = \{\text{set of books borrowed}\}$

$f(m) = \text{number of books borrowed by member } m$

Objective:

Find $\text{avg}(f(m))$

$\text{max}(f(m)), \text{min}(f(m))$

$\text{count}(m \in M \mid f(m) = 0)$

$\text{mode}(f(m))$

Algorithm:-

Input:

- book_borrow_count: Dictionary mapping book titles to total borrow counts.
- member_borrow_count: Dictionary mapping member names to total borrow counts.

Step 1: Compute Average Number of Books Borrowed by Members

1. Initialize total_borrowed = 0

2. For each member in member_borrow_count:

total_borrowed += borrow count

3. total_members = number of members

4. If total_members > 0:

average = total_borrowed / total_members

Else:

average = 0

5. Output average

🔍 **Time Complexity:** $O(n)$, where n = number of members

📦 **Space Complexity:** $O(1)$

Step 2: Find Book with Highest and Lowest Borrowings

1. If book_borrow_count is empty:

Return None for both

2. Initialize max_borrow = $-\infty$, min_borrow = ∞

3. For each book in book_borrow_count:

If borrow count > max_borrow:

max_borrow = borrow count, max_book = book

If borrow count < min_borrow:

min_borrow = borrow count, min_book = book

4. Output max_book and min_book

🔍 **Time Complexity:** $O(m)$, where m = number of books

📦 **Space Complexity:** $O(1)$

Step 3: Count Members with Zero Borrowings

1. Initialize count = 0

2. For each member in member_borrow_count:

 If borrow count == 0:

 count += 1

3. Output count

🔍 **Time Complexity:** $O(n)$, where n = number of members

📦 **Space Complexity:** $O(1)$

Step 4: Find the Most Frequently Borrowed Book (Mode)

1. If book_borrow_count is empty:

 Return None

2. Initialize frequency_map = {}

3. For each book in book_borrow_count:

 borrow_count = book_borrow_count[book]

 frequency_map[borrow_count] += 1

4. Find the borrow_count with max frequency in frequency_map → mode_count

5. For each book in book_borrow_count:

 If book_borrow_count[book] == mode_count:

Add to mode_books list

6. Output mode_books

🕒 **Time Complexity:** $O(m)$

📦 **Space Complexity:** $O(k)$, where k = number of unique borrow counts (typically $k \ll m$)

✓ Summary Table

Operation	Time Complexity	Space Complexity
Average books borrowed by members	$O(n)$	$O(1)$
Book with highest/lowest borrowings	$O(m)$	$O(1)$
Count members with zero borrowings	$O(n)$	$O(1)$
Most frequently borrowed book (mode)	$O(m)$	$O(k)$

Test Cases:

TestCase	Borrow Data	Expected Output
1	{"Alice": 2, "Bob": 0, "Charlie": 5}	Avg: 2.33, Max: 5, Min: 0, Zero Count: 1, Mode: 0/2/5 (any with freq=1)
2	{"A": 1, "B": 1, "C": 0, "D": 1}	Avg: 0.75, Max: 1, Min: 0, Zero Count: 1, Mode: 1
3	{}	Handle empty input gracefully (no division by zero or errors)

Conclusion:

In this assignment, we created a Python program to manage library borrowing records efficiently. The program calculated the average books borrowed by members, identified the most and least borrowed books, counted members with zero borrowings, and determined the most frequently borrowed book. Using Python dictionaries and built-in functions, we achieved clear, concise, and optimized solutions. We also analyzed the time and space complexities of each operation to ensure scalability.

Questions:

1. What data structures are used in this program and why?
2. What is the difference between `max()` and manually finding the maximum value?
3. Why do we use Counter from the collections module for finding the mode?

Experiment No: 02	
Title Search Operations on Customer Account IDs using Linear and Binary Search.	
Date of performance:	Date of Submission:
Class: SE	Roll No.:
Signature:	

Experiment No. 2

Title:

Search Operations on Customer Account IDs using Linear and Binary Search.

Objective:

- To implement **Linear Search** to find an account ID in a list.
- To implement **Binary Search** for efficient lookup in a sorted list.
- To compare the **efficiency** (time complexity) of both search techniques.
- To understand the conditions under which binary search is preferable.

Problem Statement:

In an e-commerce system, customer account IDs are stored in a list. Write a program to implement the following operations:

1. **Linear Search:** Check if a particular customer account ID exists in the list.
2. **Binary Search:** Improve the search efficiency by implementing binary search to find if a customer account ID exists (on a sorted list).

Theory:

- **Linear Search** works by sequentially checking each element in the list. It can be applied to both sorted and unsorted data structures. Its time complexity is **$O(n)$** , where n is the number of elements.
- The linear search is a sequential search, which uses a loop to step through an array, starting with the first element. It compares each element with the value being searched for, and stops when either the value is found or the end of the array is encountered. If the value being searched is not in the array, the algorithm will Unsuccessfully search to the end of the array. Since the array elements are stored in linear order searching the element in the linear order make it easy and efficient. The search may be successful or unsuccessfully. That is, if the required element is found then the search is successful otherwise it is unsuccessful.
- **Binary Search** requires a sorted list and works by repeatedly dividing the search interval in half. It is significantly faster for large datasets, with a time complexity of **$O(\log n)$** . It compares the middle element of the interval to the target and narrows the interval accordingly.

- Binary search is an searching algorithm which is used to find element from the sorted list”
- Algorithm can be applied only on sorted data.
- Position of mid element calculated using $\text{Mid} = (\text{lower} + \text{upper}) / 2$. Given element is compared with middle element of the list.
- If $\text{key} = \text{mid}$ then element is found
- Otherwise list divide into two part. ($\text{key} < \text{mid}$) ($> \text{mid}$)
- First to $\text{mid}-1$ or $\text{mid}+1$ to last.
- If given element is less than middle element then
 - continue searching in first part (first to $\text{mid}-1$)
 - otherwise searching is second part ($\text{mid}+1$ to last).

Repeat above step still element is found.

Mathematical Model:

Let:

- I = input list of customer IDs
- S = ID to be searched
- Output = {found, not found}
- Functions:
 - $\text{Linear}(I, S) \rightarrow O(n)$
 - $\text{Binary}(\text{sorted}(I), S) \rightarrow O(\log n)$

Time and Space Complexity:

Method	Time Complexity	Space Complexity
Linear Search	$O(n)$	$O(1)$
Binary Search	$O(\log n)$	$O(1)$

Algorithm

Linear Search:

1. Input: List of account IDs and the ID to search.
2. Iterate through each element in the list.
3. If an element matches the search ID, return found.
4. If end of list is reached, return not found.

Binary Search:

1. Input: Sorted list of account IDs and ID to search.
2. Set low = 0, high = len(list)-1.
3. While low <= high:
 - o Compute mid = (low + high) // 2.
 - o If mid element equals search ID → found.
 - o Else if search ID < mid → search in left half.
 - o Else → search in right half.
4. If not found, return not found.

Test Cases:

Test Case	Account ID List	Search ID	Expected Output - Linear	Expected Output - Binary
1	[101, 202, 303, 404]	303	Found	Found
2	[101, 202, 303, 404]	999	Not Found	Not Found
3	[]	123	Not Found	Not Found
4	[5, 10, 15, 20, 25]	5	Found	Found

Conclusion:

Both search techniques were implemented successfully. Binary search demonstrated significantly faster performance on larger, sorted data sets. Linear search was simpler but less efficient.

Questions:

1. When is Binary Search preferred over Linear Search?
2. What is the precondition for applying Binary Search?
3. What are some real-world applications of Binary Search?
4. What is the best-case time complexity of Binary Search? Why?
5. What is the worst-case time complexity of Linear Search? Why?

Experiment No: 03	
Title: Sorting Employee Salaries using Selection Sort and Bubble Sort	
Date of performance:	Date of Submission:
Class: SE	Roll No.:
Signature:	

Experiment No 3

Title:

Sorting Employee Salaries using Selection Sort and Bubble Sort

Objective:

- To implement the **Selection Sort** algorithm for sorting employee salaries in ascending order.
- To implement the **Bubble Sort** algorithm for sorting employee salaries in ascending order.
- To compare the working of Selection Sort and Bubble Sort.
- To display the **Top Five Highest Employee Salaries** after sorting.

Problem Statement:

In a company, employee salaries are stored as a list of floating-point numbers. Write a Python program to sort the employee salaries in **ascending order** using the following sorting algorithms:

1. **Selection Sort:** Sort the salaries using the Selection Sort algorithm.
2. **Bubble Sort:** Sort the salaries using the Bubble Sort algorithm.

After sorting, display the **Top Five Highest Salaries** in the company.

Theory:

Sorting is the process of arranging data in a particular order, such as ascending or descending. It helps in efficient searching, analysis, and reporting of data.

In this experiment, two simple comparison-based sorting algorithms are used:

- **Selection Sort**
- **Bubble Sort**

Selection Sort

Selection Sort repeatedly selects the **smallest element** from the unsorted portion of the list and places it at the beginning.

Working

- Divide the list into sorted and unsorted parts.
- Find the minimum element in the unsorted part.
- Swap it with the first unsorted element.
- Repeat until the list is completely sorted.

Characteristics

- Simple to implement.
- Performs fewer swaps.
- Suitable for small datasets.

Time Complexity

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

After every pass, the largest element "bubbles up" to its correct position.

Working

- Compare adjacent elements.
- Swap them if they are out of order.
- Continue until no more swaps are required.

Characteristics

- Easy to understand.
- Stable sorting algorithm.
- Efficient for nearly sorted data when optimized.

Time Complexity

- Best Case: $O(n)$ (Optimized Bubble Sort)
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Top Five Highest Salaries

Once the salaries are sorted in ascending order, the **last five elements** in the list represent the five highest salaries.

If the number of employees is less than five, display all available salaries.

Mathematical Model

Let:

- **S** = List of employee salaries
- **n** = Number of employees

Functions:

- **SelectionSort(S) → Sorted List**
- **BubbleSort(S) → Sorted List**
- **TopFive(S) → Highest Five Salaries**

Output:

- Salaries sorted in ascending order.
- Top Five Highest Salaries.

Time and Space Complexity

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Bubble Sort	$O(n)$ (Optimized)	$O(n^2)$	$O(n^2)$	$O(1)$

Algorithm

Algorithm for Selection Sort

1. Start.
2. Input the list of employee salaries.
3. For each position from 0 to $n-1$:
 - Assume the current position contains the minimum salary.
 - Search the remaining unsorted list.

- Find the actual minimum salary.
 - Swap it with the current position.
4. Repeat until the list is sorted.
 5. Display the sorted list.
 6. Display the top five highest salaries.
 7. Stop.

Algorithm for Bubble Sort

1. Start.
2. Input the list of employee salaries.
3. Compare adjacent salaries.
4. If the left salary is greater than the right salary, swap them.
5. Continue for all adjacent pairs.
6. Repeat until no swaps occur.
7. Display the sorted list.
8. Display the top five highest salaries.
9. Stop.

Test Cases

Test Case	Employee Salaries	Expected Sorted Output	Top Five Highest Salaries
1	[45000, 32000, 78000, 56000, 91000, 62000]	[32000, 45000, 56000, 62000, 78000, 91000]	91000, 78000, 62000, 56000, 45000
2	[50000, 42000, 38000]	[38000, 42000, 50000]	50000, 42000, 38000
3	[65000, 65000, 72000, 48000, 89000]	[48000, 65000, 65000, 72000, 89000]	89000, 72000, 65000, 65000, 48000
4	[]	[]	No Salaries Available

Conclusion

The employee salary list was successfully sorted in ascending order using both **Selection Sort** and **Bubble Sort** algorithms. After sorting, the **Top Five Highest Salaries** were displayed correctly. Selection Sort performs a fixed number of comparisons and fewer swaps, whereas Bubble Sort repeatedly exchanges adjacent elements until the list is sorted. Both algorithms are suitable for small datasets but become inefficient for large datasets due to their quadratic time complexity.

Questions

1. What is the difference between **Selection Sort** and **Bubble Sort**?
2. Why is Bubble Sort called a **stable sorting algorithm**?
3. What are the best, average, and worst-case time complexities of Selection Sort?
4. Under what conditions is Bubble Sort more efficient?
5. Why is Selection Sort considered an **in-place sorting algorithm**?
6. How are the **Top Five Highest Salaries** obtained after sorting in ascending order?

Experiment No: 04	
Title: Real-Time Undo/Redo System Using Stack	
Date of performance:	Date of Submission:
Class: SE	Roll No.:
Signature:	

Experiment No 4

Title:

Real-Time Undo/Redo System Using Stack

Objective:

To implement a real-time text editor simulation that supports **Undo** and **Redo** functionalities using **Stack** data structure.

Develop a program that simulates a basic text editing application using **two stacks** to manage user actions. The application must support:

Problem Statement

- Making changes to a document.
- Undoing the most recent change.
- Redoing a previously undone change.
- Displaying the current state of the document.

Theory

A **stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle. It is ideal for implementing **Undo/Redo** functionalities:

- When a user makes a change, the previous state is pushed to the **Undo Stack**.
- When the user performs **Undo**, the current state is pushed to the **Redo Stack**, and the previous state is popped from the Undo Stack.
- When the user performs **Redo**, the last undone state is restored from the Redo Stack and pushed back onto the Undo Stack.

Using two stacks:

- `undo_stack`: stores document states before each change.
- `redo_stack`: stores states that were undone and can be redone.

Algorithm

Initialization:

- `undo_stack` $\leftarrow$ empty
- `redo_stack` $\leftarrow$ empty
- `document` $\leftarrow$ ""

Make a Change (new_text):

1. Push current `document` to `undo_stack`
2. Clear `redo_stack`
3. Set `document` $\leftarrow$ `new_text`

Undo():

1. If `undo_stack` is not empty:
 - Push current `document` to `redo_stack`
 - Pop top from `undo_stack` and assign to `document`
2. Else: Print "Nothing to undo"

Redo():

1. If `redo_stack` is not empty:
 - Push current `document` to `undo_stack`
 - Pop top from `redo_stack` and assign to `document`
2. Else: Print "Nothing to redo"

Display():

- Print the current value of `document`

Test Cases

Operation	Input Text	Output / Document State
Make Change	"Hello"	Hello
Make Change	"Hello World"	Hello World
Undo		Hello
Redo		Hello World
Undo		Hello
Make Change	"New Text"	New Text
Redo		Nothing to redo (Redo stack cleared)

Conclusion

This experiment successfully demonstrates the use of **stack** data structures to implement **Undo** and **Redo** operations. It simulates how modern text editors allow users to revert and reapply changes. The use of two stacks ensures an efficient and logical approach to managing editing history.

Experiment No: 05	
Title: Real-Time Event Processing System using Queue Data Structure	
Date of performance:	Date of Submission:
Class: SE	Roll No.:
Signature:	

Experiment No. 5

Title:

Real-Time Event Processing System using Queue Data Structure.

Objective:

- To implement a Queue data structure for real-time event processing.
- To perform queue operations such as enqueue, dequeue, display, and cancel event.
- To understand the FIFO (First-In, First-Out) principle used in queue operations.
- To simulate a real-time event management system using queue data structure.

Problem Statement:

In a real-time event processing system, events are generated continuously and need to be processed in the order they arrive. Develop a program using the Queue data structure that supports the following operations:

1. **Add an Event:** Insert a new event into the queue.
2. **Process the Next Event:** Remove and process the event that has been waiting the longest.
3. **Display Pending Events:** Display all events currently waiting for processing.
4. **Cancel an Event:** Remove a specified event from the queue if it has not yet been processed.

Theory:

A **Queue** is a linear data structure that follows the **FIFO (First-In, First-Out)** principle. The first element inserted into the queue is the first one to be removed. Queue operations are commonly used in scheduling systems, printer management, operating systems, network packet processing, and event-driven applications.

The two primary operations of a queue are:

- **Enqueue:** Inserts an element at the rear of the queue.
- **Dequeue:** Removes an element from the front of the queue.

Additional operations include:

- **Display:** Shows all elements currently present in the queue.
- **Cancel Event:** Removes a specific event from the queue before it is processed.

Queue Characteristics

- Follows **FIFO** order.
- Insertion takes place at the **rear**.
- Deletion takes place from the **front**.
- Efficient for scheduling and event management systems.

Queue Operations

Enqueue (Add Event):

- Adds a new event at the rear of the queue.

Dequeue (Process Event):

- Removes the oldest event from the front of the queue.

Display Events:

- Displays all pending events in their processing order.

Cancel Event:

- Searches for the specified event and removes it if present in the queue.

Mathematical Model

Let

- **E** = Set of incoming events
- **Q** = Queue containing pending events
- **A** = Add Event (Enqueue)
- **P** = Process Event (Dequeue)
- **D** = Display Pending Events
- **C** = Cancel Event

Then,

- **A(E) → Q**
- **P(Q) → Process Front Event**
- **D(Q) → Display All Pending Events**
- **C(Event) → Remove Event if Exists**

Output:

- Updated queue after every operation.

Time and Space Complexity

Operation	Time Complexity	Space Complexity
Add Event (Enqueue)	$O(1)$	$O(1)$
Process Event (Dequeue)	$O(1)$	$O(1)$
Display Events	$O(n)$	$O(1)$
Cancel Event	$O(n)$	$O(1)$

Where **n** is the number of events currently in the queue.

Algorithm

Algorithm for Add Event (Enqueue)

1. Start.
2. Read the new event.
3. Insert the event at the rear of the queue.
4. Display confirmation message.
5. Stop.

Algorithm for Process Next Event (Dequeue)

1. Start.
2. Check whether the queue is empty.
3. If empty, display "**No events to process.**"
4. Otherwise,
 - Remove the front event.
 - Process the event.
5. Stop.

Algorithm for Display Pending Events

1. Start.
2. Check whether the queue is empty.
3. If empty, display "**No pending events.**"
4. Otherwise,
 - Traverse the queue from front to rear.
 - Display each event.

5. Stop.

Algorithm for Cancel Event

1. Start.
2. Read the event name/ID to cancel.
3. Search the queue for the event.
4. If found,
 - Remove the event from the queue.
 - Display **"Event Cancelled Successfully."**
5. Otherwise,
 - Display **"Event Not Found."**
6. Stop.

Test Cases

Test Case	Initial Queue	Operation	Expected Output
1	[]	Add "Event A"	Queue = [Event A]
2	[A, B, C]	Process Event	Queue = [B, C]
3	[A, B, C]	Display Events	A B C
4	[A, B, C]	Cancel B	Queue = [A, C]
5	[]	Process Event	Queue Underflow / No events to process
6	[A, B]	Cancel D	Event Not Found

Conclusion

The Queue data structure was successfully implemented to simulate a real-time event processing system. The system efficiently handled event insertion, processing, display, and cancellation while maintaining the FIFO principle. Queue proved to be an ideal data structure for managing events in the order of their arrival, making it suitable for real-time applications such as operating systems, customer service systems, printer queues, and network packet scheduling.

Questions

1. What is the FIFO principle in a Queue?
2. Why is Queue preferred for real-time event processing systems?
3. What is the difference between Queue and Stack?
4. What are the time complexities of Enqueue and Dequeue operations?
5. What happens when a Dequeue operation is performed on an empty queue?

Experiment No: 06	
Title: Call Center Queue Management System using Queue Data Structure	
Date of performance:	Date of Submission:
Class: SE	Roll No.:
Signature:	

Experiment No 6

Title:

Call Center Queue Management System using Queue Data Structure

Objective:

- To implement a Queue data structure for managing incoming customer calls.
- To simulate call center operations using the FIFO (First-In, First-Out) principle.
- To perform queue operations such as adding calls, answering calls, viewing the queue, and checking queue status.
- To understand the practical application of queues in real-time customer service systems.

Problem Statement:

A call center receives incoming customer calls, where each call is assigned a unique **Customer ID** and a **Call Time (in minutes)**. The calls are answered strictly in the order they are received.

Develop a program using the Queue data structure that supports the following operations:

1. **addCall(customerID, callTime):** Add a new customer call to the queue with the customer ID and estimated call time.
2. **answerCall():** Answer and remove the first customer call from the queue.
3. **viewQueue():** Display all customer calls currently waiting in the queue without removing them.
4. **isEmpty():** Check whether the call queue is empty.

Theory:

A **Queue** is a linear data structure that follows the **FIFO (First-In, First-Out)** principle, where the first element inserted into the queue is the first element removed. Queue is one of the most commonly used data structures in real-time systems such as call centers, ticket booking systems, printer scheduling, operating systems, and network packet management.

In a call center, customers are served in the exact order in which they call. Every new incoming call is added to the **rear** of the queue, while the customer who has been waiting the longest is answered from the **front** of the queue.

Queue Operations

addCall(customerID, callTime):

- Inserts a new customer call at the rear of the queue.

answerCall():

- Removes and processes the customer call at the front of the queue.

viewQueue():

- Displays all customer calls currently waiting in the queue.

isEmpty():

- Checks whether the queue contains any pending customer calls.

Characteristics of Queue

- Follows **FIFO (First-In-First-Out)** principle.
- Insertion is performed at the rear.
- Deletion is performed from the front.
- Efficient for scheduling and waiting-line applications.
- Provides fair servicing of requests based on arrival order.

Mathematical Model

Let:

- **C** = Set of customer calls
- **Q** = Queue of pending calls
- **ID** = Customer ID
- **T** = Call Time (minutes)

Functions:

- **addCall(ID, T) → Q**
- **answerCall(Q) → Remove Front Call**
- **viewQueue(Q) → Display Pending Calls**
- **isEmpty(Q) → {True, False}**

Output:

- Updated queue after each operation.

Time and Space Complexity

Operation	Time Complexity	Space Complexity
addCall()	$O(1)$	$O(1)$
answerCall()	$O(1)$	$O(1)$
viewQueue()	$O(n)$	$O(1)$
isEmpty()	$O(1)$	$O(1)$

Where **n** is the number of customer calls waiting in the queue.

Algorithm

Algorithm for addCall(customerID, callTime)

1. Start.
2. Input Customer ID and Call Time.
3. Create a new call record.
4. Insert the call at the rear of the queue.
5. Display confirmation message.
6. Stop.

Algorithm for answerCall()

1. Start.
2. Check whether the queue is empty.
3. If empty,
 - Display "**No calls waiting.**"
4. Otherwise,
 - Remove the customer call from the front of the queue.
 - Display the answered Customer ID and Call Time.
5. Stop.

Algorithm for viewQueue()

1. Start.
2. Check whether the queue is empty.
3. If empty,
 - Display "**Call Queue is Empty.**"

4. Otherwise,
 - Traverse the queue from front to rear.
 - Display Customer ID and Call Time of each waiting call.
5. Stop.

Algorithm for isEmpty()

1. Start.
2. Check whether the queue contains any customer calls.
3. If queue size equals zero,
 - Return **True**.
4. Otherwise,
 - Return **False**.
5. Stop.

Test Cases

Test Case	Initial Queue	Operation	Expected Output
1	Empty	addCall(101, 5)	Queue = [(101, 5)]
2	[(101,5), (102,3)]	answerCall()	Queue = [(102,3)]
3	[(101,5), (102,3)]	viewQueue()	Displays Customer IDs 101 and 102 with Call Times
4	Empty	isEmpty()	True
5	[(101,5)]	isEmpty()	False
6	Empty	answerCall()	"No calls waiting."

Conclusion

The Call Center Queue Management System was successfully implemented using the Queue data structure. The program efficiently managed incoming customer calls according to the **FIFO (First-In-First-Out)** principle. Operations such as adding calls, answering calls, viewing pending calls, and checking queue status were performed successfully. This implementation demonstrates the practical use of queues in customer service applications, ensuring fair and efficient handling of customer requests.

Questions

1. What is the FIFO principle, and why is it important in a call center?
2. Why is Queue the most suitable data structure for managing customer calls?
3. What are the time complexities of **addCall()** and **answerCall()** operations?
4. What happens if **answerCall()** is performed on an empty queue?
5. What is the difference between a Queue and a Stack?

Experiment No: 07	
Title: Implementation of Hash Table using Division Method and Chaining	
Date of performance:	Date of Submission:
Class: SE	Roll No.:
Signature:	

Experiment No 7

Title:

Implementation of Hash Table using Division Method and Chaining

Objective:

- To implement a Hash Table of size 10 using the Division Method as the hash function.
- To resolve collisions using the Chaining technique.
- To perform insertion, searching, and deletion of key-value pairs in the hash table.
- To understand the concept of hashing and collision resolution techniques.

Problem Statement:

A hash table is an efficient data structure used for storing and retrieving data. Implement a **Hash Table of size 10** using the **Division Method** as the hash function. In case of collisions, use the **Chaining** technique to store multiple key-value pairs at the same index.

The program should support the following operations:

1. **Insert(key, value):** Insert a key-value pair into the hash table.
2. **Search(key):** Search for the value associated with the given key.
3. **Delete(key):** Delete the specified key-value pair from the hash table.

Theory:

A **Hash Table** is a data structure that stores data in the form of **key-value pairs**. It provides fast insertion, searching, and deletion operations by mapping keys to specific indices using a **hash function**.

A **Hash Function** converts a given key into an array index where the corresponding value is stored.

Division Method

The Division Method computes the hash index using the formula:

Hash Index = Key mod Table Size
$$\text{Hash Index} = \text{Key} \bmod \text{Table Size}$$

For a hash table of size **10**,

$h(\text{key}) = \text{key} \bmod 10$

Example:

Key Hash Index

15 5

25 5

35 5

Since all three keys map to the same index, a **collision** occurs.

Collision

A collision occurs when two or more keys produce the same hash index.

Chaining

Chaining is a collision resolution technique where each index of the hash table stores a linked list (or list) of all key-value pairs that hash to the same index.

Example:

Index 5

15 → 1500

↓

25 → 2500

↓

35 → 3500

Operations

Insert

- Compute the hash index.
- Insert the key-value pair into the linked list at that index.

Search

- Compute the hash index.
- Traverse the chain and locate the required key.

Delete

- Compute the hash index.
- Search for the key.
- Remove the corresponding node from the chain.

Mathematical Model

Let

- **K** = Set of Keys
- **V** = Set of Values
- **HT** = Hash Table of size 10

Hash Function:

$$h(k) = k \bmod 10$$

Functions:

- **Insert(K,V) → HT**
- **Search(K) → Value / Not Found**
- **Delete(K) → Updated HT**

Output:

- Updated hash table after each operation.

Time and Space Complexity

Operation Average Case Worst Case Space Complexity

Insert	$O(1)$	$O(n)$	$O(n)$
Search	$O(1)$	$O(n)$	$O(1)$
Delete	$O(1)$	$O(n)$	$O(1)$

Where **n** is the number of key-value pairs stored in the hash table.

Algorithm

Algorithm for Insert(key, value)

1. Start.
2. Input key and value.
3. Compute the hash index:

index = key mod 10

4. Insert the key-value pair into the linked list (chain) at the computed index.
5. Display success message.
6. Stop.

Algorithm for Search(key)

1. Start.
2. Input the key.
3. Compute:

index = key mod 10

4. Traverse the linked list at that index.
5. If key is found,
 - Display the associated value.
6. Otherwise,
 - Display "Key Not Found."
7. Stop.

Algorithm for Delete(key)

1. Start.
2. Input the key.
3. Compute:

index = key mod 10

4. Traverse the linked list.
5. If key is found,
 - Delete the corresponding node.
 - Display "Key Deleted Successfully."
6. Otherwise,
 - Display "Key Not Found."
7. Stop.

Test Cases

Test Case	Operation	Input	Expected Output
1	Insert	(15, "A")	Inserted at index 5
2	Insert	(25, "B")	Collision handled using chaining
3	Search	25	Value = "B"
4	Search	40	Key Not Found
5	Delete	15	Key Deleted Successfully
6	Delete	100	Key Not Found
7	Insert	(35, "C")	Added to chain at index 5

Conclusion

The Hash Table of size **10** was successfully implemented using the **Division Method** as the hash function and **Chaining** for collision resolution. The operations of insertion, searching, and deletion were performed efficiently. Chaining effectively handled collisions by storing multiple key-value pairs at the same hash index, ensuring that data could still be accessed correctly. Hash tables provide fast average-case performance and are widely used in databases, compilers, caching systems, symbol tables, and dictionaries.

Questions

1. What is a Hash Table?
2. What is the purpose of a hash function?
3. Explain the Division Method used in hashing.
4. What is a collision in a hash table?
5. Why is chaining used for collision resolution?
6. What are the average-case and worst-case time complexities of searching in a hash table?

Experiment No: 08	
Title: Implementation of Hash Table using Division Method and Linear Probing	
Date of performance:	Date of Submission:
Class: SE	Roll No.:
Signature:	

Experiment No 8

Title:

Implementation of Hash Table using Division Method and Linear Probing

Objective:

- To implement a Hash Table of fixed size using the Division Method as the hash function.
- To resolve collisions using the Linear Probing technique.
- To perform insertion, searching, deletion, and display operations on the hash table.
- To understand open addressing and collision resolution using linear probing.

Problem Statement:

Design and implement a **Hash Table of fixed size** using the **Division Method** as the hash function. In case of a collision, resolve it using the **Linear Probing** technique.

The program should support the following operations:

1. **Insert(key):** Insert a key into the hash table.
2. **Search(key):** Search for a given key in the hash table.
3. **Delete(key):** Delete a key from the hash table.
4. **Display():** Display the contents of the hash table.

Theory:

A **Hash Table** is a data structure used to store data efficiently by mapping keys to indices using a **hash function**. It provides fast insertion, searching, and deletion operations.

The **Division Method** is one of the simplest and most commonly used hash functions. It computes the hash index as:

Hash Index = Key mod Table Size

$$\text{Hash Index} = \text{Key} \bmod \text{Table Size}$$

For a table of size **m**,

$$h(\text{key}) = \text{key} \bmod m$$

Example (Table Size = 10):

Key Hash Index

15 5

25 5

35 5

Since multiple keys generate the same hash index, a **collision** occurs.

Collision

A collision occurs when two or more keys are mapped to the same index in the hash table.

Linear Probing

Linear Probing is an **open addressing** collision resolution technique. If the computed index is already occupied, the algorithm checks the next available location sequentially until an empty slot is found.

Probe sequence:

$$(h(\text{key}) + i) \bmod m$$

where,

- **$h(\text{key})$** = Hash value
- **$i = 0, 1, 2, 3, \dots$**
- **m** = Table Size

Example:

Suppose the table size is **10**.

Insert keys:

15 → Index 5

25 → Collision at 5 → Stored at 6

35 → Collision at 5 and 6 → Stored at 7

Thus,

Index : Value

5 : 15

6 : 25

7 : 35

Operations

Insert

- Compute the hash index.
- If the location is empty, insert the key.
- Otherwise, apply linear probing until an empty slot is found.

Search

- Compute the hash index.
- Compare the key.
- If not found, continue probing until the key is found or an empty slot is encountered.

Delete

- Locate the key using linear probing.
- Delete the key by marking the slot appropriately (e.g., as deleted).

Display

- Traverse the hash table and display all occupied and empty slots.

Mathematical Model

Let:

- **K** = Set of Keys
- **HT** = Hash Table of size **m**
- **m** = Fixed Table Size

Hash Function:

$$h(k) = k \bmod m$$

Linear Probing Function:

$$\text{Index} = (h(k) + i) \bmod m$$

where,

- $i = 0, 1, 2, \dots$

Functions:

- **Insert(K) → HT**
- **Search(K) → Found / Not Found**
- **Delete(K) → Updated HT**
- **Display() → Current Hash Table**

Output:

- Updated hash table after every operation.

Time and Space Complexity

Operation Average Case Worst Case Space Complexity

Insert	$O(1)$	$O(n)$	$O(1)$
Search	$O(1)$	$O(n)$	$O(1)$
Delete	$O(1)$	$O(n)$	$O(1)$
Display	$O(n)$	$O(n)$	$O(1)$

Where **n** is the size of the hash table.

Algorithm

Algorithm for Insert(key)

1. Start.
2. Input the key.
3. Compute:

index = key mod table_size

4. If the location is empty,
 - Insert the key.
5. Otherwise,

- Apply linear probing:
 - Check the next location.
 - Repeat until an empty slot is found.
- 6. Insert the key.
- 7. Display success message.
- 8. Stop.

Algorithm for Search(key)

1. Start.
2. Input the key.
3. Compute:

 $\text{index} = \text{key} \bmod \text{table_size}$
4. Compare the key with the current slot.
5. If found,
 - Display "**Key Found.**"
6. Otherwise,
 - Continue linear probing.
7. If an empty slot is encountered,
 - Display "**Key Not Found.**"
8. Stop.

Algorithm for Delete(key)

1. Start.
2. Input the key.
3. Compute:

 $\text{index} = \text{key} \bmod \text{table_size}$
4. Search the key using linear probing.
5. If found,
 - Delete the key (mark the slot as deleted).
6. Otherwise,
 - Display "**Key Not Found.**"
7. Stop.

Algorithm for Display()

1. Start.
2. Traverse all indices of the hash table.
3. Display each index and its corresponding key.
4. If the slot is empty, display **Empty**.

5. Stop.

Test Cases

Test Case Operation Input			Expected Output
1	Insert	15	Inserted at index 5
2	Insert	25	Collision at index 5, inserted at index 6
3	Insert	35	Collision resolved, inserted at index 7
4	Search	25	Key Found
5	Search	99	Key Not Found
6	Delete	25	Key Deleted Successfully
7	Display	—	Displays complete hash table

Conclusion

The Hash Table was successfully implemented using the **Division Method** and **Linear Probing** for collision resolution. The operations of insertion, searching, deletion, and display were performed efficiently. Linear probing effectively resolved collisions by searching sequentially for the next available slot. Although it provides good average-case performance, excessive collisions can lead to clustering, which may reduce efficiency. Hash tables with linear probing are widely used in dictionaries, symbol tables, database indexing, compiler design, and cache implementations.

Questions

1. What is a Hash Table?
2. What is the Division Method in hashing?
3. What is a collision in a hash table?
4. Explain the Linear Probing technique with an example.
5. What is the difference between **Linear Probing** and **Chaining**?

Experiment No: 09	
Title: Graph Traversal using Breadth First Search (BFS) and Depth First Search (DFS)	
Date of performance:	Date of Submission:
Class: SE	Roll No.:
Signature:	

Experiment No 9

Title:

Graph Traversal using Breadth First Search (BFS) and Depth First Search (DFS)

Objective:

- To represent a graph using an **Adjacency Matrix** for Depth First Search (DFS).
- To represent a graph using an **Adjacency List** for Breadth First Search (BFS).
- To implement **DFS** and **BFS** graph traversal algorithms.
- To understand the working of graph traversal techniques and their real-world applications.

Problem Statement:

Consider a particular area in your city. Let the popular locations **A, B, C, D, E, F, G** represent the nodes of a graph. If there exists a route between two locations, it is represented as an edge between the corresponding nodes.

Write a program to determine the sequence in which these locations are visited starting from node **A** using:

1. **Breadth First Search (BFS)** using an **Adjacency List** representation.
2. **Depth First Search (DFS)** using an **Adjacency Matrix** representation.

Theory:

A **Graph** is a non-linear data structure consisting of **vertices (nodes)** and **edges** connecting these vertices. Graphs are widely used to represent networks such as road maps, social networks, communication systems, and transportation networks.

Graphs can be represented using:

- **Adjacency Matrix**
- **Adjacency List**

1. Adjacency Matrix

An adjacency matrix is a two-dimensional array of size $V \times V$, where V is the number of vertices.

- $\text{Matrix}[i][j] = 1$ if an edge exists between vertex i and j .

- $\text{Matrix}[i][j] = 0$ otherwise.

It requires $O(V^2)$ memory.

2. Adjacency List

An adjacency list stores all adjacent vertices of every vertex in the form of linked lists (or lists).

It requires $O(V + E)$ memory, where:

- V = Number of vertices
- E = Number of edges

It is more space-efficient for sparse graphs.

Breadth First Search (BFS)

Breadth First Search is a graph traversal algorithm that visits all neighboring vertices before moving to the next level. It uses a **Queue** data structure.

Working:

- Start from the source vertex.
- Visit all adjacent vertices.
- Add unvisited adjacent vertices to the queue.
- Repeat until the queue becomes empty.

Characteristics:

- Uses Queue.
- Traverses level by level.
- Finds the shortest path in an unweighted graph.

Time Complexity:

$O(V+E)O(V+E)O(V+E)$

Depth First Search (DFS)

Depth First Search is a graph traversal algorithm that explores as far as possible along each branch before backtracking. It uses **Recursion** or a **Stack**.

Working:

- Start from the source vertex.
- Visit an adjacent unvisited vertex.
- Continue until no unvisited adjacent vertex exists.
- Backtrack and continue traversal.

Characteristics:

- Uses Stack (or Recursion).
- Explores depth before breadth.

Time Complexity:

$O(V+E)$ $O(V+E)$ $O(V+E)$

Mathematical Model

Let

- $G = (V, E)$

Where,

- V = Set of vertices (locations)
- E = Set of edges (roads/routes)

Input:

- Graph G
- Starting Vertex = A

Functions:

- $DFS(G, A) \rightarrow$ Depth First Traversal
- $BFS(G, A) \rightarrow$ Breadth First Traversal

Output:

- Sequence of visited locations.

Time and Space Complexity

Algorithm	Time Complexity	Space Complexity
DFS	$O(V + E)$	$O(V)$
BFS	$O(V + E)$	$O(V)$
Adjacency Matrix	$O(V^2)$ Space	$O(V^2)$
Adjacency List	$O(V + E)$ Space	$O(V + E)$

Where:

- **V** = Number of vertices
- **E** = Number of edges

Algorithm

Algorithm for DFS (Using Adjacency Matrix)

1. Start.
2. Create the adjacency matrix.
3. Mark all vertices as unvisited.
4. Select the starting vertex (A).
5. Mark the current vertex as visited.
6. Display the current vertex.
7. Traverse all adjacent vertices.
8. If an adjacent vertex is unvisited,
 - Recursively call DFS for that vertex.
9. Continue until all reachable vertices are visited.
10. Stop.

Algorithm for BFS (Using Adjacency List)

1. Start.
2. Create the adjacency list.
3. Mark all vertices as unvisited.
4. Insert the starting vertex (A) into the queue.
5. Mark it as visited.
6. While the queue is not empty:
 - Remove the front vertex.
 - Display it.

- Visit all unvisited adjacent vertices.
 - Insert them into the queue.
7. Repeat until the queue becomes empty.
 8. Stop.

Example Graph

Vertices:

A, B, C, D, E, F, G

Edges:

A-B
A-C
B-D
B-E
C-F
E-G
F-G

Adjacency Matrix (for DFS)

	A	B	C	D	E	F	G
A	0	1	1	0	0	0	0
B	1	0	0	1	1	0	0
C	1	0	0	0	0	1	0
D	0	1	0	0	0	0	0
E	0	1	0	0	0	0	1
F	0	0	1	0	0	0	1
G	0	0	0	0	1	1	0

Adjacency List (for BFS)

A → B → C

B → A → D → E

C → A → F

D → B

E → B → G

F → C → G

G → E → F

Expected Traversals

Starting Vertex = A

DFS Traversal:

A → B → D → E → G → F → C

BFS Traversal:

A → B → C → D → E → F → G

Test Cases

Test Case Starting Vertex Expected DFS Expected BFS

1	A	ABDEGFC	ABCDEFG
2	B	BACFGED	BADECGF
3	C	CABDEGF	CAFBGDE
4	Single Vertex	A	A

Conclusion

The graph was successfully represented using an **Adjacency Matrix** for **Depth First Search (DFS)** and an **Adjacency List** for **Breadth First Search (BFS)**. DFS explored the graph by visiting vertices as deep as possible before backtracking, whereas BFS visited vertices level by level using a queue. Both traversal techniques were implemented successfully and demonstrated efficient graph exploration. These algorithms are

widely used in applications such as route planning, social networks, web crawling, network broadcasting, and pathfinding.

Questions

1. What is a graph? Explain its components.
2. What is the difference between an **Adjacency Matrix** and an **Adjacency List**?
3. Why is **DFS** generally implemented using recursion or a stack?
4. Why does **BFS** require a queue?
5. Compare **DFS** and **BFS** with suitable examples.
6. What are the time and space complexities of DFS and BFS?

Experiment No: 10	
Title: Implementation of Binary Search Tree (BST) Operations	
Date of performance:	Date of Submission:
Class: SE	Roll No.:
Signature:	

Experiment No 10

Title:

Implementation of Binary Search Tree (BST) Operations

Objective:

- To implement a Binary Search Tree (BST).
- To perform insertion, deletion, searching, and display operations on a BST.
- To understand the properties and working of a Binary Search Tree.
- To analyze the efficiency of BST operations.

Problem Statement:

A **Binary Search Tree (BST)** is a hierarchical data structure in which each node has at most two children. The left subtree of a node contains values smaller than the node, while the right subtree contains values greater than the node.

Design and implement a Binary Search Tree that supports the following operations:

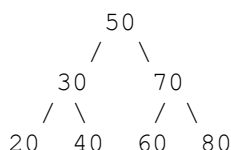
1. **Insert(key):** Insert a new node into the BST.
2. **Search(key):** Search for a specified key in the BST.
3. **Delete(key):** Delete a node from the BST.
4. **Display():** Display the BST using inorder traversal.

Theory:

A **Binary Search Tree (BST)** is a special type of binary tree that satisfies the following properties:

- Every node contains a key.
- All keys in the left subtree are smaller than the root node.
- All keys in the right subtree are greater than the root node.
- Both left and right subtrees are themselves BSTs.

Example:



Advantages of BST

- Efficient searching, insertion, and deletion.
- Maintains data in sorted order.
- Supports dynamic data management.

BST Operations

1. Insertion

To insert a new key:

- Compare the key with the root node.
- If smaller, move to the left subtree.
- If greater, move to the right subtree.
- Repeat until an empty position is found.

2. Searching

To search for a key:

- Compare the key with the root.
- If equal, the key is found.
- If smaller, search in the left subtree.
- If greater, search in the right subtree.

3. Deletion

Deletion in BST has three cases:

Case 1: Node with No Children (Leaf Node)

Simply remove the node.

Before:

```
  50
 /
20
```

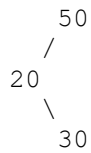
After:

```
  50
 /
-
```

Case 2: Node with One Child

Replace the node with its child.

Before:



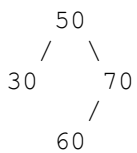
After:

50

Case 3: Node with Two Children

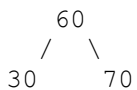
Replace the node with its inorder successor (smallest node in the right subtree) or inorder predecessor.

Before:



Delete 50

After:

**4. Display (Inorder Traversal)**

Inorder Traversal:

- Traverse Left Subtree
- Visit Root
- Traverse Right Subtree

The output of inorder traversal in a BST is always in **sorted order**.

Mathematical Model

Let:

- **T** = Binary Search Tree
- **N** = Set of nodes
- **K** = Key value

Functions:

- **Insert(T, K) → Updated BST**
- **Search(T, K) → Found / Not Found**
- **Delete(T, K) → Updated BST**
- **Display(T) → Sorted Traversal**

Output:

- Updated BST after each operation.

Time and Space Complexity

Operation	Best Case	Average Case	Worst Case
Insert	$O(\log n)$	$O(\log n)$	$O(n)$
Search	$O(\log n)$	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(\log n)$	$O(n)$
Inorder Traversal	$O(n)$	$O(n)$	$O(n)$

Where:

- **n** = Number of nodes in the BST.

Algorithm

Algorithm for Insert(key)

1. Start.
2. Create a new node.
3. If the tree is empty,
 - Make the node the root.
4. Otherwise,
 - Compare the key with the current node.
 - If key < current node, move to the left child.
 - Else move to the right child.
5. Insert the node at the appropriate position.
6. Stop.

Algorithm for Search(key)

1. Start.
2. Begin at the root node.
3. Compare the key with the current node.
4. If equal,
 - Display "**Key Found**".
5. If key < current node,
 - Search left subtree.
6. Otherwise,
 - Search right subtree.
7. If NULL is reached,
 - Display "**Key Not Found**".
8. Stop.

Algorithm for Delete(key)

1. Start.
2. Search for the node.
3. If node is not found,
 - Display "**Key Not Found**".
4. Otherwise:
 - If node is a leaf, delete it.
 - If node has one child, replace it with its child.
 - If node has two children:
 - Find inorder successor.
 - Replace node value with successor value.
 - Delete successor node.
5. Stop.

Algorithm for Display (Inorder Traversal)

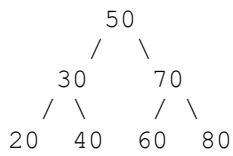
1. Start.
2. Traverse the left subtree.
3. Visit the root node.
4. Traverse the right subtree.
5. Display nodes in sorted order.
6. Stop.

Example BST

Insert the keys:

50, 30, 70, 20, 40, 60, 80

Resulting BST:



Test Cases

Test Case Operation			Input	Expected Output
1	Insert	50		Node inserted
2	Insert	30		Node inserted
3	Search	40		Key Found
4	Search	90		Key Not Found
5	Delete	20		Node deleted successfully
6	Delete	50		Root node deleted and replaced appropriately
7	Display	Inorder Traversal	20 30 40 50 60 70 80	

Sample Execution

Initial BST

50 30 70 20 40 60 80

Inorder Traversal

20 30 40 50 60 70 80

Search Key = 40

Key Found

Delete Key = 20

Node Deleted Successfully

Updated Inorder Traversal

30 40 50 60 70 80

TAE PUNE

Experiment No: 11	
Title: Construction of Expression Tree from Prefix Expression and Non-Recursive Postorder Traversal	
Date of performance:	Date of Submission:
Class: SE	Roll No.:
Signature:	

ASSIGNMENT NO 11

Title:

Construction of Expression Tree from Prefix Expression and Non-Recursive Postorder Traversal.

Objective:

- To construct an **Expression Tree** from a given prefix expression.
- To perform **non-recursive postorder traversal** of the expression tree.
- To delete the entire expression tree after traversal.
- To understand the representation of arithmetic expressions using binary trees.

Problem Statement:

Construct an **Expression Tree** from the given **prefix expression** (e.g., **+-a*bc/def**). Perform the following operations:

1. Construct the expression tree from the prefix expression.
2. Traverse the expression tree using **non-recursive postorder traversal**.
3. Delete the entire expression tree and free the allocated memory.

Theory:

An **Expression Tree** is a binary tree used to represent arithmetic expressions.

- **Leaf Nodes** represent operands (variables or constants).
- **Internal Nodes** represent operators such as **+, -, *, /**.

Expression trees provide an efficient way to evaluate and manipulate arithmetic expressions.

There are three commonly used expression notations:

- **Prefix (Polish Notation):** Operator appears before operands.
- **Infix Notation:** Operator appears between operands.
- **Postfix (Reverse Polish Notation):** Operator appears after operands.

Example:

Prefix Expression:

$+-a*bc/def$

Equivalent Infix Expression:

$((a-(b*c))-(d/e))+f$

Construction of Expression Tree

To construct an expression tree from a **prefix expression**:

- Scan the prefix expression from **right to left**.
- If the symbol is an operand, create a node and push it onto the stack.
- If the symbol is an operator:
 - Pop two nodes from the stack.
 - Create a new operator node.
 - Make the first popped node the left child and the second popped node the right child.
 - Push the new node back onto the stack.
- Continue until the expression is completely scanned.
- The remaining node in the stack becomes the root of the expression tree.

Postorder Traversal

Postorder traversal visits nodes in the following order:

Left Subtree → Right Subtree → Root

For an expression tree, postorder traversal produces the **postfix expression**.

Since recursion is not allowed, a **stack** is used to perform the traversal iteratively.

Tree Deletion

To delete the tree:

- Visit every node.
- Delete child nodes first.
- Finally delete the root node.

This ensures proper memory deallocation.

Mathematical Model

Let:

- **P** = Prefix Expression
- **T** = Expression Tree

Functions:

- **Construct(P) → T**
- **Postorder(T) → Postfix Expression**
- **Delete(T) → Empty Tree**

Output:

- Constructed Expression Tree
- Postorder Traversal
- Empty Tree after deletion

Time and Space Complexity

Operation	Time Complexity	Space Complexity
Tree Construction	$O(n)$	$O(n)$
Postorder Traversal	$O(n)$	$O(n)$
Tree Deletion	$O(n)$	$O(h)$

Where:

- **n** = Number of nodes
- **h** = Height of the tree

Algorithm

Algorithm for Constructing Expression Tree

1. Start.
2. Read the prefix expression.
3. Scan the expression from right to left.
4. If the current symbol is an operand:
 - Create a node.
 - Push it onto the stack.
5. Else if the symbol is an operator:
 - Pop two nodes from the stack.
 - Create a new operator node.

- Make the first popped node the left child.
 - Make the second popped node the right child.
 - Push the new node onto the stack.
6. Repeat until the entire expression is scanned.
 7. The remaining node in the stack is the root.
 8. Stop.

Algorithm for Non-Recursive Postorder Traversal

1. Start.
2. Initialize two stacks.
3. Push the root node onto Stack 1.
4. While Stack 1 is not empty:
 - Pop a node from Stack 1.
 - Push it onto Stack 2.
 - Push its left child (if any) onto Stack 1.
 - Push its right child (if any) onto Stack 1.
5. Pop all nodes from Stack 2 and display them.
6. Stop.

Algorithm for Deleting the Tree

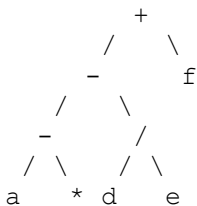
1. Start.
2. Traverse the tree in postorder.
3. Delete each node after visiting its children.
4. Continue until all nodes are deleted.
5. Set the root pointer to NULL.
6. Stop.

Example

Prefix Expression

`+-a*bc/def`

Expression Tree



$$\begin{array}{cc} / & \backslash \\ b & c \end{array}$$

Postorder Traversal (Postfix Expression)

abc*-de/-
f+

Combined:

abc*-de/-f+

Test Cases

Test Case Prefix Expression Expected Postorder

1	+AB	AB+
2	*+ABC	AB+C*
3	+-a*bc/def	abc*-de/-f+
4	++AB/CD	AB+CD/-

Conclusion

The expression tree was successfully constructed from the given **prefix expression**. A **non-recursive postorder traversal** using stacks generated the corresponding postfix expression correctly. Finally, the entire tree was deleted to release the allocated memory. Expression trees are widely used in compilers, expression evaluation, syntax analysis, and arithmetic expression processing.

Questions

1. What is an expression tree?
2. What are the different types of expression notations?
3. Why is an expression tree considered a binary tree?
4. Why is the prefix expression scanned from **right to left** during tree construction?
5. Which traversal of an expression tree produces the postfix expression?
6. Why are stacks used for non-recursive postorder traversal?